MATERIAL DIDÁTICO DO PROFESSOR • AULA 7

Dominando o Ciclo do CRUD no Quarkus

Desmistificando Beans, Inversão de Controle (IoC), Validação Declarativa com Records do Java 21, Persistência Saudável e Arquitetura Global de Exceções.

Java 21 | Quarkus 3.39+ | Hibernate Panache | Bean Validation | CDI (Arc) | REST Enterprise

1 Desmistificando o Conceito de "Bean" no Java

A palavra **Bean** assusta muitos estudantes por ser utilizada em contextos diferentes. Vamos entender de forma simples sua evolução histórica e seu significado no Quarkus:

> ☕ **Por que "Bean"? (A Curiosidade Histórica)**
> Java é o nome de uma ilha indonésia famosa pelo cultivo de café (origem do símbolo da xícara). Em inglês, grão de café é *Coffee Bean*. Os criadores da linguagem batizaram os blocos fundamentais reutilizáveis de **Beans (grãos de café)**.

Termo	O que é de fato?	Exemplo Prático
JavaBean Clássico	Uma classe com atributos privados, getters/setters e construtor vazio. Apenas uma "sacola de dados".	`public class User { private String email; public String getEmail(){ ... } }`
CDI Bean (Quarkus)	Um objeto **gerenciado pelo framework**. Você não dá `new`; o Quarkus cria, guarda e injeta com `@Inject`.	`@ApplicationScoped public class UserService { ... }`
Bean Validation	Especificação do Java para carimbar regras de validação direto nos atributos do objeto via anotações.	`@NotBlank @Email private String email;`
@Valid	O "porteiro" do Controller que aciona o motor de validação antes de executar o método.	`public Response create(@Valid UserForm form)`

2 Inversão de Controle (IoC) e Proxies: Por que NUNCA dar "new"?

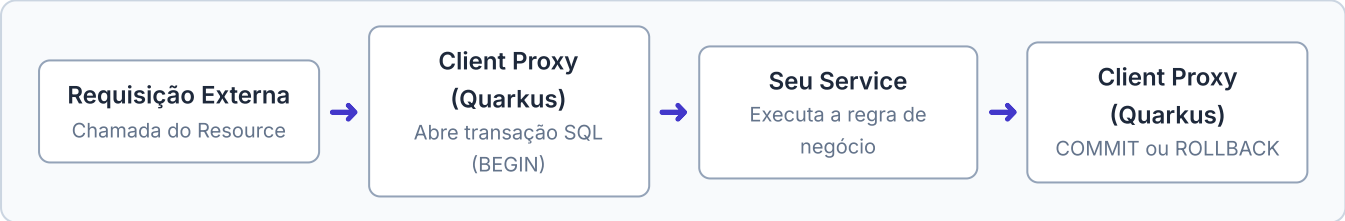
Quando colocamos `@ApplicationScoped` no `UserService` e injetamos com `@Inject`, o Quarkus assume a responsabilidade sobre aquele objeto. Mas o que isso muda na prática?

> 🚗 **A Analogia do Uber vs. Comprar um Carro**
> **Dar `new`:** É comprar o carro, trocar o óleo, calibrar pneus e estacionar toda vez que precisa ir à padaria.
> **O Quarkus no controle:** É pedir um Uber. Você não precisa saber quem construiu o carro ou como ele funciona por dentro; você apenas solicita a corrida e a central entrega o serviço pronto.

> ⚠️ **O Maior Perigo: O `@Transactional` morre se você der `new`!**
> O Java puro não entende a anotação `@Transactional`. Se você instanciar `new UserService()` manualmente, o método rodará como código comum: **nenhuma transação será aberta no banco de dados**, o commit não acontecerá e falhas não farão rollback.

Como o Quarkus resolve isso? Através de Client Proxies:

O Quarkus nunca injeta a sua classe original pura. Ele cria uma **"casca invisível" (Proxy)** ao redor dela:



3 Validação Declarativa (Bean Validation) com Records do Java 21

Validações manuais com dezenas de `if (campo == null || campo.isBlank())` poluem a regra de negócio. O padrão moderno utiliza anotações declarativas em **Records** imutáveis do Java 21:

```java
// dto/user/UserForm.java - Dados que CHEGAM do cliente
package dto.user;

import jakarta.validation.constraints.*;

public record UserForm(
    @NotBlank(message = "O e-mail é obrigatório")
    @Email(message = "Formato de e-mail inválido")
    String email,

    @NotBlank(message = "A senha é obrigatória")
    @Size(min = 6, message = "A senha deve conter no mínimo 6 caracteres")
    String password
) {}
```

```java
// dto/user/UserDTO.java - Dados que SAEM para o cliente (SENHA BLINDADA)
package dto.user;

import java.time.LocalDateTime;
import java.util.UUID;

public record UserDTO(
    UUID id,
    String email,
    LocalDateTime createdAt
) {}
```

> 💡 **Por que separar Form e DTO?**
> O **Form** recebe a senha aberta do usuário para ser validada e criptografada. O **DTO** é a bandeja de entrega segura: **nunca devolva a entidade JPA inteira**, pois o hash da senha ou dados internos vazariam no JSON de resposta.

4 Persistência Saudável no Panache: "Só chamo persist e já era?"

Nos esqueletos iniciais, é comum encontrar chamadas como `repository.create(userForm)`. No Quarkus Panache:

- O método `create()` **não existe** no Panache (ele usa `persist()`).
- O Repositório **não deve conhecer** DTOs ou Forms. Ele é responsável exclusivamente por Entidades do banco.

Por que NÃO devemos apenas chamar `repository.persist()` sem validações?
Se chamarmos `persist` diretamente e o e-mail já existir, o banco estourará uma `ConstraintViolationException` de SQL pura, devolvendo um horrível erro **500 Internal Server Error** para o cliente.

O Ciclo dos 5 Passos de uma Persistência Saudável:




```java
// Implementação no UserService.java
@Transactional
public UserDTO create(UserForm form) {
    // 1. Validação de Unicidade prévia
    userRepository.findByEmail(form.email()).ifPresent(u -> {
        throw new ConflictException("E-mail já cadastrado: " + form.email());
    });

    // 2. Mapeamento de Entrada
    User user = userMapper.toEntity(form);

    // 3. Enriquecimento de Dados
    user.passwordHash = SecurityUtils.hashPassword(form.password());

    // 4. Persistência Transacional
    userRepository.persist(user);

    // 5. Retorno Seguro via DTO
    return userMapper.toDTO(user);
}
```

5 Tratamento Global de Exceções: Eliminando o "try/catch" do Controller

Em vez de sujar todos os endpoints com blocos `try/catch` genéricos que mascaram falhas, a arquitetura moderna centraliza o tratamento em um **Global Exception Handler**.

Organização do Módulo `exception/`:

- `exception/dto/` : Contém `ErrorResponseDTO` e `ValidationErrorDTO` (formato padronizado do JSON de erro).
- `exception/custom/` : Exceções com semântica de negócio (`ResourceNotFoundException` , `ConflictException` , `BusinessException`).
- `exception/handler/` : O interceptador `GlobalExceptionHandler` com anotações `@ServerExceptionMapper` .

```java
// exception/dto/ErrorResponseDTO.java - Resposta JSON Oficial de Erro
package exception.dto;

import java.time.LocalDateTime;
import java.util.List;

public record ErrorResponseDTO(
    int status,
    String error,
    String message,
    String path,
    LocalDateTime timestamp,
    List<ValidationErrorDTO> fieldErrors
) {
    public ErrorResponseDTO(int status, String error, String message, String path) {
        this(status, error, message, path, LocalDateTime.now(), null);
    }
}
```

```java
// exception/handler/GlobalExceptionHandler.java - Interceptador Global
package exception.handler;

import exception.custom.*;
import exception.dto.*;
import io.quarkus.resteasy.reactive.server.ServerExceptionMapper;
import jakarta.validation.ConstraintViolationException;
import jakarta.ws.rs.core.*;

public class GlobalExceptionHandler {

    @ServerExceptionMapper
    public Response handleNotFound(ResourceNotFoundException ex, @Context UriInfo uriInfo)
        var err = new ErrorResponseDTO(404, "Not Found", ex.getMessage(), uriInfo.getPath()
        return Response.status(Response.Status.NOT_FOUND).entity(err).build();
    }

    @ServerExceptionMapper
    public Response handleConflict(ConflictException ex, @Context UriInfo uriInfo) {
        var err = new ErrorResponseDTO(409, "Conflict", ex.getMessage(), uriInfo.getPath())
        return Response.status(Response.Status.CONFLICT).entity(err).build();
    }
}
```

O Resultado: Resource Limpo, Seguro e Semântico

```java
// resource/UserResource.java - ZERO blocos try/catch!
@POST
public Response create(@Valid UserForm form) {
    UserDTO created = userService.create(form);
    return Response.status(Response.Status.CREATED).entity(created).build();
}
```

🎯 **Checklist Rápido de Sala de Aula:**

1. Classes de Service e Mapper com `@ApplicationScoped`?
2. Repositories estendendo `PanacheRepositoryBase<T, UUID>`?
3. Métodos de escrita anotados com `@Transactional`?
4. Parâmetro de entrada do Resource com `@Valid`?
5. Exceções de negócio capturadas pelo `GlobalExceptionHandler`?